

Blockchains & Cryptocurrencies

Consensus & Towards Bitcoin



Instructor: Matthew Green
Johns Hopkins University - Spring 2026

Many slides based on NBFMG

Housekeeping

- Readings, NBFMG
- Please keep up with the readings!
- A2B is out
 - See the course syllabus and the Github

News?

News?



r/CryptoCurrency • 6h ago
gdscripto



Strategy Makes Another Bitcoin Purchase as Unrealized Losses Mount

GENERAL-NEWS



News?

MoneyWatch

Purported ransom note for Savannah Guthrie's mom demanded payment in bitcoin. Could it be tracked?

By Aimee Picchi

February 6, 2026 / 5:38 PM EST / CBS News

 Add CBS News on Google

The disappearance of "Today" show co-host Savannah Guthrie's mother, Nancy Guthrie, includes a tantalizing clue: an apparent ransom note that demanded payment in bitcoin.

The note, which is being taken seriously by investigators, included a deadline of 5 p.m. Thursday, although it didn't specify a time zone. If the payment wasn't made, the note specified another deadline of Monday, investigators said Thursday.

The demand for bitcoin raises questions about whether the cryptocurrency could be used to hide a kidnapper's identity, given the financial product's semi-anonymous nature. Experts tell CBS News that law enforcement officials can track down information about the people or organizations behind crypto transactions, increasing the chances that the ransom demand could produce possible leads in the Guthrie case.

News?



Monero 51% Attack: Why AI Protocol Qubic Says It'll 'Help' the Privacy Chain



Ryan Gladwin

August 12, 2025 • 5 min read



Today

- We're going to talk about “consensus”
- What the heck is consensus, how do you accomplish it, what's the point?
- Then we'll evolve towards Bitcoin



Review: hashes /
signatures

Reminder: hash functions

- Take as input an arbitrary-length string
- Output a (shorter) fixed-size string

Cryptographic hash function security:

Reminder: hash functions

- Take as input an arbitrary-length string
- Output a (shorter) fixed-size string

Cryptographic hash function security:

- Collision-resistant
- Pre-image resistant
- “Random oracle”-like (for some cases)

Reminder: hash functions

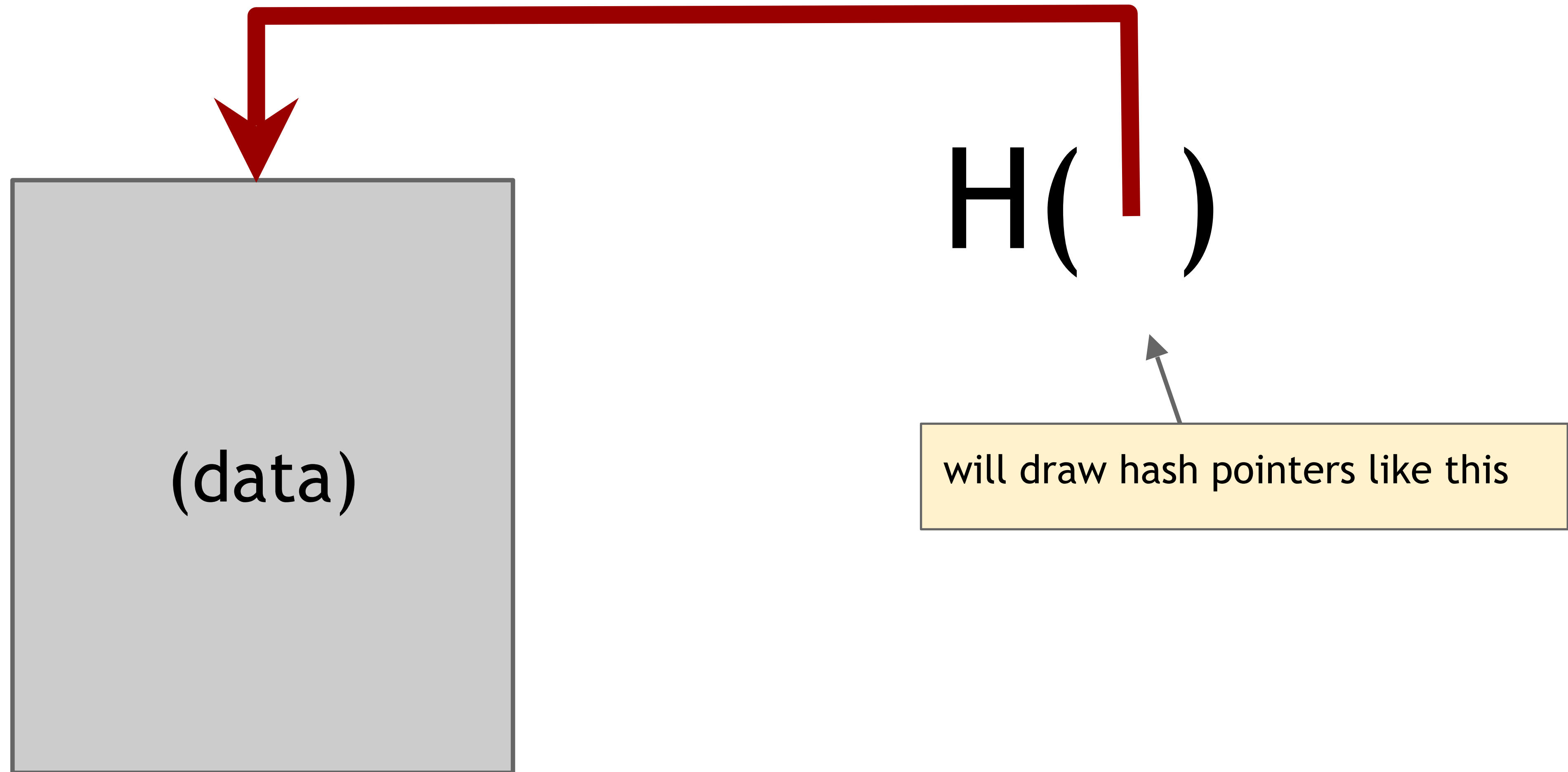
- Some examples (current+historical):
 - MD5 ✗
 - SHA1 ✗
 - SHA2 family ✓
SHA256, SHA512, SHA384 (also SHA224 🙄)
 - SHA3 (AKA “Keccak”) ✓
 - Blake2 ✓

Hash pointer

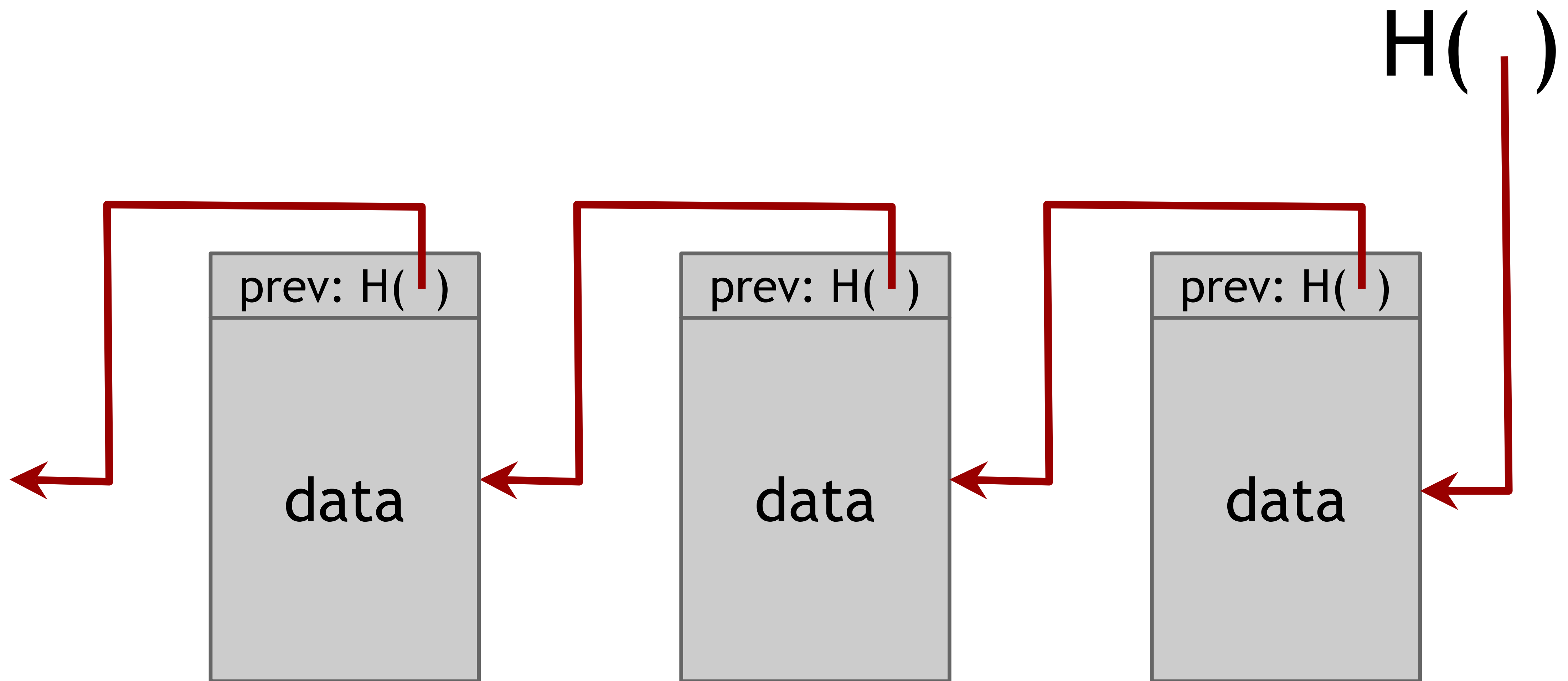
- pointer to where some info is stored, *and*
- cryptographic hash of the info

If we have a hash pointer, we can

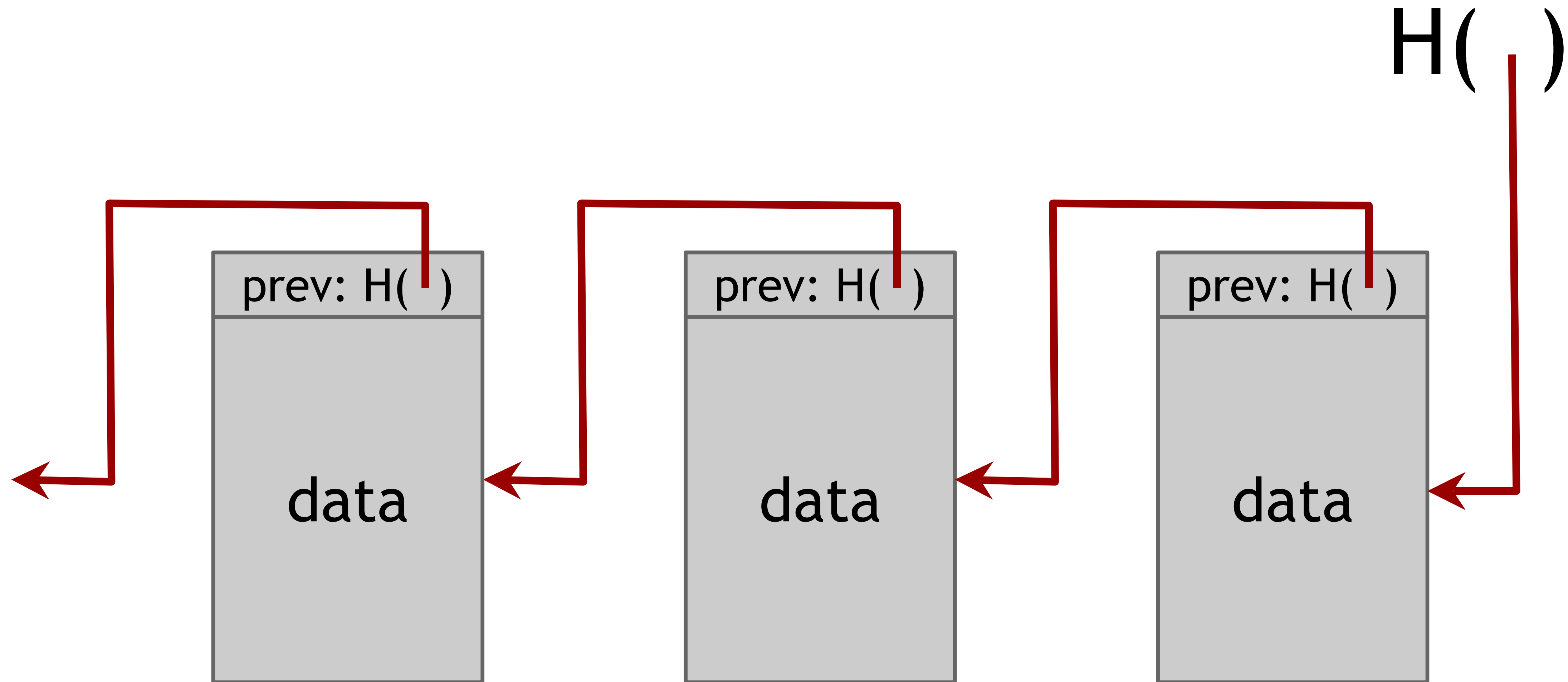
- ask to get the info back, and
- verify that it hasn't changed



Building data structures with hash pointers

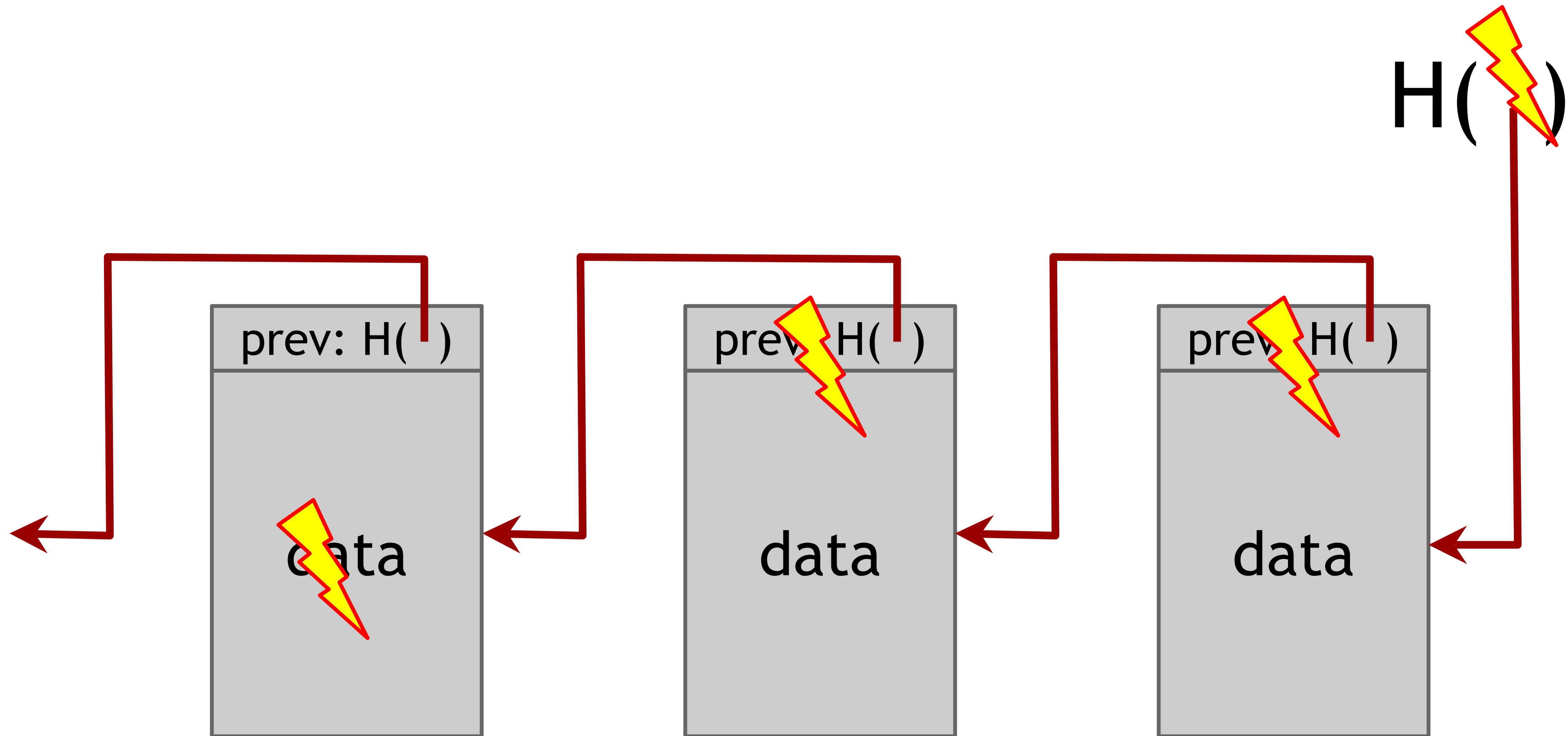


Linked list with hash pointers = “Blockchain”



use case: tamper-evident log

detecting tampering



use case: tamper-evident log

Digital signatures

Security parameter

- $(sk, pk) \leftarrow \text{keygen}(1^k)$

sk: secret signing key

pk: public verification key

} randomized
algorithm

- $\text{sig} \leftarrow \text{sign}(sk, \text{message})$

} Typically
randomized

- $\text{isValid} \leftarrow \text{verify}(pk, \text{message}, \text{sig})$

Requirements for signatures

- Correctness: “valid signatures verify”
 - $\text{verify}(\text{pk}, \text{message}, \text{sign}(\text{sk}, \text{message})) == \text{true}$
- Unforgeability under chosen-message attacks (UF-CMA): “can’t forge signatures”
 - adversary who knows pk , and gets to see signatures on messages of his choice, can’t produce a verifiable signature on another message

Review: cash problems

- **Double spending**

- To capture double spending you need an online (networked) party that must be trusted

- **Authentication / Authentication**

- How do I prove that I am the owner of currency & thus authorized to transact with it?

- **Origin/Issuance**

- How is new currency created?

Partial approach

- Let's not dispense with our centralized approach (just yet)
- We will, however, reduce the number of our assumptions
- Our new assumption is that there is a **centralized** party that can maintain a ledger
- This centralized party also can create (“mint”) new currency and assign it to be owned by users



GoofyCoin



Goofy can create new coins

signed by pk_{Goofy}

CreateCoin [uniqueCoinID]

New coins belong to me.



A coin's owner can spend it.

signed by pk_{Goofy}
Pay to $pk_{\text{Alice}} H()$

signed by pk_{Goofy}
CreateCoin [uniqueCoinID]

Alice owns it
now.



The recipient can pass on the coin again.

signed by pk_{Alice}
Pay to $pk_{\text{Bob}} : H()$

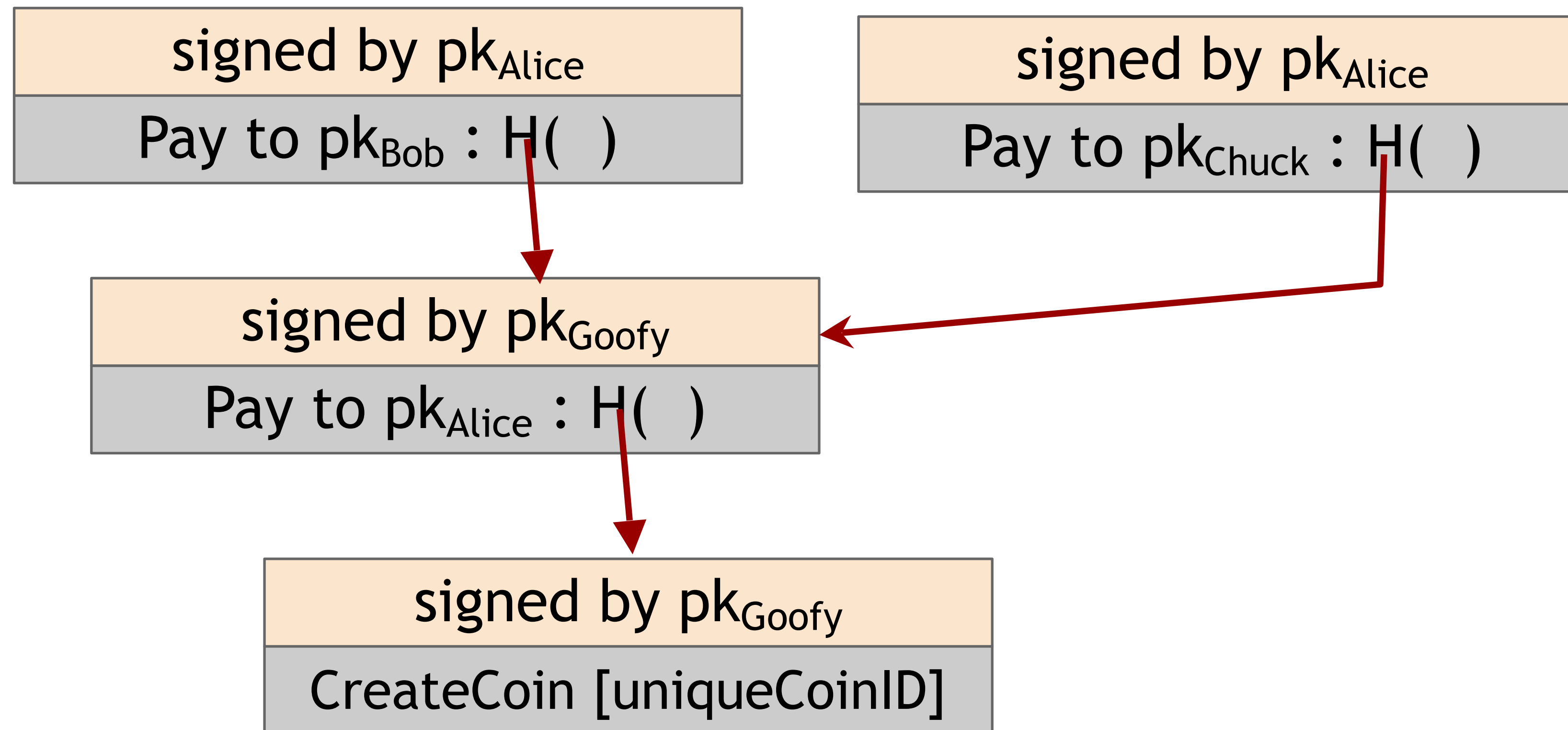
signed by pk_{Goofy}
Pay to $pk_{\text{Alice}} : H()$

signed by pk_{Goofy}
CreateCoin [uniqueCoinID]

Bob owns it now.



double-spending attack



double-spending attack

This is the main design
challenge in digital currency

How do we solve this?

How do we solve this?

- Simplest answer: send all transactions to an atomic, append-only **centralized** ledger
- Have the ledger provide a definite ordering for transactions
 - If two transactions conflict, simply disallow the later one
- No TX is valid unless the ledger has “approved” and ordered it



ScroogeCoin

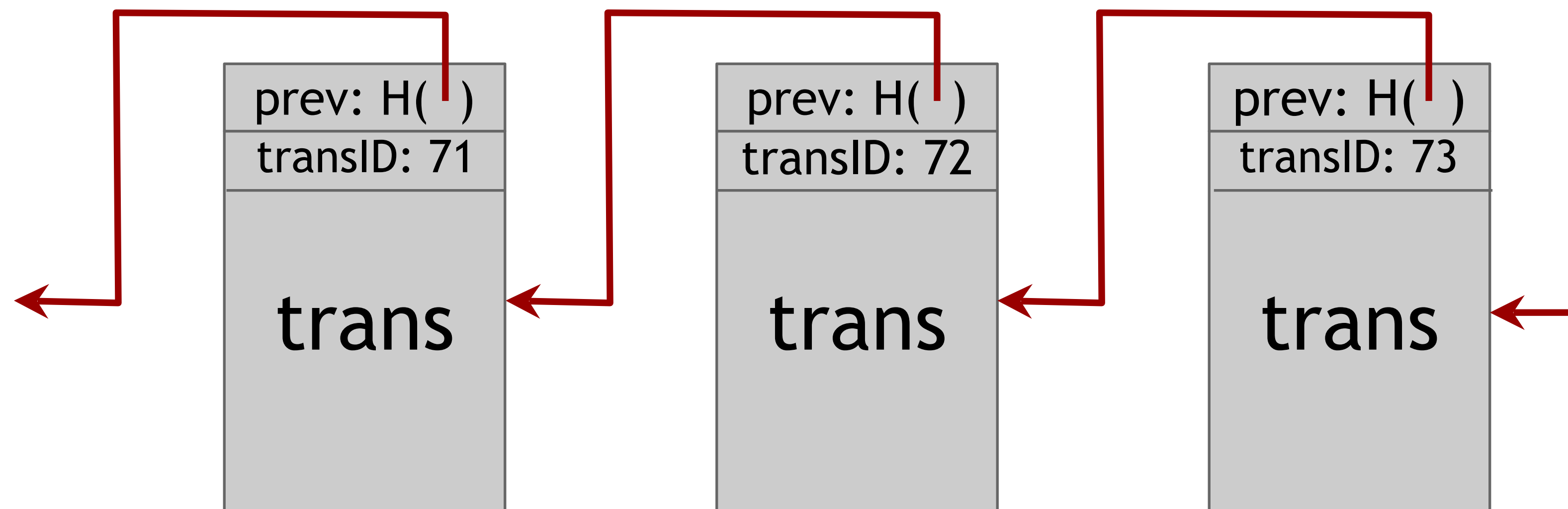


Scrooge publishes a history of all transactions in an “append-only” ledger

Implement the ledger using a block chain, signed by Scrooge



$H()$
Sig



optimization: put multiple transactions in the same block

CreateCoins transaction creates new coins

Valid, because I said so.

transID: 73 type:CreateCoins		
coins created		
<i>num</i>	<i>value</i>	<i>recipient</i>
0	3.2	0x...
1	1.4	0x...
2	7.1	0x...

← coinID 73(0)

← coinID 73(1)

← coinID 73(2)

signature



CreateCoins transaction creates new coins

Valid, because I said so.

transID: 73 type:CreateCoins		
coins created		
<i>num</i>	<i>value</i>	<i>recipient</i>
0	3.2	0x...
1	1.4	0x...
2	7.1	0x...

These are
public keys!

← coinID 73(0)

← coinID 73(1)

← coinID 73(2)

signature



PayCoins transaction consumes (and destroys) some coins,
and creates new coins of the same total value

transID: 73 type:PayCoins		
consumed coinIDs: 68(1), 42(0), 72(3)		
coins created		
<i>num</i>	<i>value</i>	<i>recipient</i>
0	3.2	0x...
1	1.4	0x...
2	7.1	0x...

Valid if:

- consumed coins valid,
- not already consumed,
- total value out = total value in, and
- signed by owners of all consumed coins

One signature for
each consumed coin

signatures

Immutable coins

Coins can't be transferred, subdivided, or combined.

But: you can get the same effect by using transactions

- to subdivide: create new transaction

- consume your coin

- pay out two new coins to yourself

Don't worry, I'm
honest.



Crucial question:

Can we descroogify the
currency, and operate without
any central, trusted party?

Don't worry, I'm honest.



Crucial question:

Can we descroogify the currency, and operate without any central, trusted party?

Related question:

Why do we need to do this?

Centralization vs. Decentralization

- **Competing paradigms that underlie many technologies**
- Decentralized $\neq$ Distributed
(as in distributed system) but we'll often use them as synonyms

Centralization vs. Decentralization

- Examples:
 - email?
 - WWW?
 - DNS?
- What about software development?

Aspects of decentralization in Bitcoin

1. Who maintains the ledger?
2. Who has authority over which transactions are valid?
3. Who creates (and obtains) new bitcoins?
4. Who determines how the rules change?
5. How do these coins acquire monetary value?

Aspects of decentralization in Bitcoin

Peer-to-peer network:

- open to anyone, low barrier to entry
- high node churn (nodes can come and go)

Mining:

- open to anyone, but inevitable concentration of power
- often seen as undesirable

Updates to software:

- core developers trusted by community, have great power